WeatherNauts

Adnane GARAB Massy MERAKEB Hakim ROHIMUN

Agent : FAAH

Sommaire

I. Data Cleaning

II. Feature Engineering

III. Δ-Decomposition

IV. Modèle & Optimisation

Data Cleaning

Contexte : Les données brutes nécessitent un prétraitement rigoureux avant tout entraînement. Filtrage, correction physique et gestion des valeurs manquantes.

I. **Filtrage géographique**
Restriction aux 20 villes du concours, suppression des observations hors périmètre.

II. **Clip physique**
Correction des valeurs aberrantes par projection sur les bornes physiques réelles : $x \leftarrow \min(\max(x, x_{\min}), x_{\max})$.

III. **Visibilité**
Colonne `visibility` fixée à 0, 100% NaN dans le dataset.

IV. **Déduplication**
Suppression des doublons sur la clé `(city_name, timestamp)`.

V. **Imputation des NaN**
Stratégie en cascade par ville : interpolation linéaire, *forward fill*, *backward fill*, médiane globale en dernier recours.

VI. **Encodage temporel**
Extraction de `hour` et `month` depuis le timestamp UTC.

VII. **Export**
Fichiers `weather_{année}_clean.csv` générés pour 2020–2026.

Sources : WMO Global Climate Extremes, Météo-France, Met Office, AEMET, Copernicus Climate Change Service.

Data Cleaning : Code

data.cleaning.py

```python
import pandas as pd
import numpy as np

PHYSICAL_LIMITS = {
    'temperature':    (-45,  50),
    # Mouthe -41C / Sicile +48.8C
    'wind_speed':     (0,    75),
    # Fraserburgh 1989
    'rain':           (0,   200),
    # Valencia 2024
    'humidity':       (0,   100),
    # borne physique
    'clouds':         (0,   100),
    # borne physique
    'snow':           (0,    40),
    # Capracotta 2015
    'wind_direction': (0,   360),
    # borne geometrique
}
numeric_cols = [
    'temperature', 'rain',
    'wind_speed', 'wind_direction',
    'humidity', 'clouds', 'snow'
]

def clean_year(year):
    df = pd.read_csv(f"weather_{year}.csv")
    df = df[df['city_name'].isin(cities_20)]

    if 'visibility' in df.columns:
        df['visibility'] = 0
```

```python
    df['timestamp'] = pd.to_datetime(
        df['timestamp'], utc=True)

    df = df.sort_values(
        ['city_name','timestamp']
    ).drop_duplicates(
        subset=['city_name','timestamp'])
    df['hour']  = df['timestamp'].dt.hour
    df['month'] = df['timestamp'].dt.month

    for col, (lo, hi) in PHYSICAL_LIMITS.items():
        if col in df.columns:
            df[col] = df[col].clip(lo, hi)

    df[numeric_cols] = (
        df.groupby('city_name')[numeric_cols]
        .transform(lambda x:
            x.interpolate(method='linear')
             .ffill().bfill())
    )

    for col in numeric_cols:
        df[col] = df[col].fillna(
            df[col].median())

    df.to_csv(
        f"weather_{year}_clean.csv", index=False)

for year in range(2020, 2027):
    clean_year(year)
```

Sources : WMO, Météo-France, Met Office, AEMET, Copernicus Climate Change Service.

Feature Engineering

Contexte : Le tenseur brut $X \in \mathbb{R}^{20\times 24\times 8}$ contient 3 840 valeurs. On en extrait $\phi(X) \in \mathbb{R}^{98}$.

I. Valeurs courantes

$$\boldsymbol{x}_t = \left(x^{(f)}_{\text{Paris},\,t}\right)_{f=1}^{8} \in \mathbb{R}^8$$

```
x_last = X[16, -1, :]
```

II. Stats fenêtrées

Pour $w \in \{6, 24\}$, $f \in \{T, W, R, H, C\}$:

$$\mu_{f,w} = \frac{1}{w}\sum_{k=1}^{w} x^{(f)}_{\text{Paris},\,t-k}$$

$$\sigma_{f,w} = \sqrt{\frac{1}{w}\sum_{k=1}^{w}\left(x^{(f)}_{\text{Paris},\,t-k} - \mu_{f,w}\right)^2}$$

```
for w in [6, 24]:
    mu = X[16, -w:, f].mean()
    s  = X[16, -w:, f].std()
```

III. Lags temporels

Pour $\ell \in \{1, 2, 3, 6, 12, 23\}$:

$$\delta^{(f)}_{\ell} = x^{(f)}_{\text{Paris},\,t} - x^{(f)}_{\text{Paris},\,t-\ell}$$

```
for l in [1,2,3,6,12,23]:
    d = X[16,-1,f] - X[16,-1-l,f]
```

IV. Voisins géographiques

Soit d_i la distance de la ville i à Paris :

$$w_i = \frac{1}{1 + d_i/\sigma}, \qquad \sigma = \text{med}(d)$$

$$\text{spread}^{(f)} = x^{(f)}_{\text{Paris},t} - \sum_{i\neq \text{P}} w_i\, x^{(f)}_{i,t}$$

```
w = 1/(1 + dist/np.median(dist))
w /= w.sum()
spread = X[16,-1,f] - w @ X[nb,-1,f]
```

V. Composantes du vent

Soit W_t la vitesse et θ_t la direction :

$$u_t = W_t \sin\theta_t, \qquad v_t = W_t \cos\theta_t$$

```
theta = X[16,-1,3] * np.pi / 180
u = X[16,-1,2] * np.sin(theta)
v = X[16,-1,2] * np.cos(theta)
```

VI. Advection thermique

Soit a_j la projection du vent de j vers Paris :

$$a_j = u_j \cos\alpha_j + v_j \sin\alpha_j$$

$$\text{adv}_T = \sum_{j\in\mathcal{V}} a_j \cdot \left(x_{j,t}^{(T)} - x_{\text{Paris},t}^{(T)}\right)$$

```
dx = coords_paris - coords[j]
alpha = np.arctan2(dx[1], dx[0])
a_j = u[j]*np.cos(alpha) + v[j]*np.sin(
      alpha)
adv_T = sum(a_j*(X[j,-1,0]-X[16,-1,0]))
```

VII. Accélération du vent

Dérivée seconde discrète :

$$\ddot{W}_t = x_{\text{Paris},t}^{(W)} - 2x_{\text{Paris},t-1}^{(W)} + x_{\text{Paris},t-2}^{(W)}$$

```
acc = X[16,-1,2] - 2*X[16,-2,2] + X
      [16,-3,2]
```

VIII. Laplacien spatial

Diffusion thermique discrète :

$$\nabla^2 T_{\text{Paris}} \approx \frac{1}{|\mathcal{V}|} \sum_{j\in\mathcal{V}} \frac{x_{j,t}^{(T)} - x_{\text{Paris},t}^{(T)}}{d_j^2}$$

$$\frac{\partial T}{\partial t} = \alpha \nabla^2 T \implies \nabla^2 T > 0 \Rightarrow T \nearrow$$

```
lap = np.mean((X[nb,-1,0]-X[16,-1,0])/dist
      **2)
```

IX. Convergence du vent

Cohérence directionnelle des vents voisins :

$$\bar{R} = \sqrt{\left(\frac{1}{|\mathcal{V}|}\sum_j \sin\theta_j\right)^2 + \left(\frac{1}{|\mathcal{V}|}\sum_j \cos\theta_j\right)^2}$$

$$\text{conv} = 1 - \bar{R} \in [0, 1]$$

```
thetas = X[nb,-1,3] * np.pi/180
R = np.sqrt(np.mean(np.sin(thetas))**2
          + np.mean(np.cos(thetas))**2)
conv = 1 - R
```

Prédire la variation Δy

La série $\{y_t\}$ est non-stationnaire : elle porte une tendance diurne et saisonnière à variance élevée. Plutôt que d'apprendre une surface complexe, on prédit la *correction* entre l'état actuel et l'état futur, ce qui stabilise la cible et améliore la convergence du modèle.

Preuve formelle. Soit $\rho_6 = \mathrm{Corr}(y_t,\, y_{t-6})$ le coefficient d'autocorrélation à 6h. La variance du résidu s'écrit :

$$\sigma_\Delta^2 = \mathrm{Var}(y_{t+6} - y_t) = 2\sigma_y^2\big(1 - \rho_6\big)$$

Puisque $\rho_6 \in (0, 1)$ pour toute série météo, on obtient :

$$\sigma_\Delta^2 \ll \sigma_y^2$$

La variance cible est strictement réduite, ce qui implique un meilleur rapport signal/bruit et une convergence plus rapide du gradient boosting.

Reconstruction :

$$\hat{T}_{t+6} = T_t + \hat{\Delta}_T, \qquad \hat{W}_{t+6} = \max\Big(0,\, W_t + \hat{\Delta}_W\Big), \qquad \hat{R}_{t+6} = \max\Big(0,\, \hat{R}\Big)$$

HistGradientBoostingRegressor

Le gradient boosting construit un ensemble de M arbres de décision en séquence, chaque arbre corrigeant les erreurs du précédent :

$$F_M(\boldsymbol{f}) = \sum_{m=1}^{M} \eta \cdot h_m(\boldsymbol{f}), \qquad h_m = \arg\min_h \sum_{i=1}^{N} \ell(y_i,\ F_{m-1}(\boldsymbol{f}_i) + h(\boldsymbol{f}_i))$$

où η est le learning rate et ℓ la fonction de perte.

Pourquoi ce choix ?

Propriété	Justification
Données tabulaires	Le gradient boosting domine empiriquement sur ce type de données
NaN natifs	Aucun prétraitement supplémentaire requis
Régularisation	`l2_regularization` contrôle l'overfitting sur les cibles Δy
Zéro dépendance	Contrairement à LightGBM, aucune lib système externe (`libgomp`)
Robustesse	Moins sensible aux outliers que les méthodes linéaires

Trois instances indépendantes, trois fonctions de perte :

$$\mathcal{L}_T = \frac{1}{N}\sum_i |\Delta T_i - \hat{\Delta} T_i|, \qquad \mathcal{L}_W = \frac{1}{N}\sum_i |\Delta W_i - \hat{\Delta} W_i|, \qquad \mathcal{L}_R = \frac{1}{N}\sum_i |R_i - \hat{R}_i|$$

Optuna : TPE Sampler

Optuna est un framework d'optimisation bayésienne qui remplace la recherche par grille. Il utilise le TPE pour modéliser la distribution des bons hyperparamètres et orienter intelligemment l'exploration. À chaque essai, il met à jour ses estimations et converge vers la région optimale de l'espace $\mathcal{X}$:

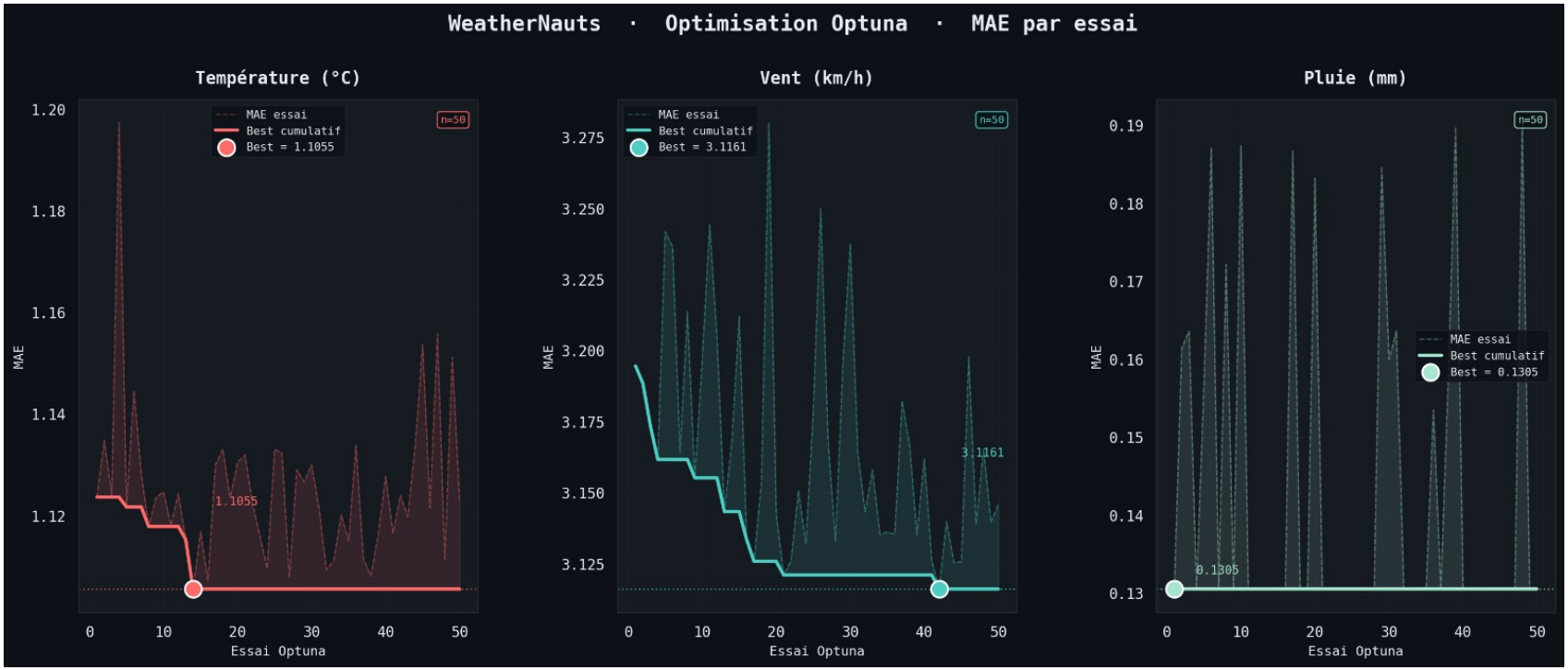
$$x^* = \arg\max_{x \in \mathcal{X}} \frac{\ell(x)}{g(x)}$$

où $\ell(x)$ modélise les bons essais et $g(x)$ les mauvais. Train sur 2020–2025, validation sur 2026.

```
def objective(trial):
    params = {
        "loss":    trial.suggest_categorical("loss",
                   ["absolute_error","huber","poisson"]),# perte par variable
        "lr":      trial.suggest_float("lr",
                   0.005, 0.15, log=True),               # vitesse apprentissage
        "max_iter": trial.suggest_int("max_iter",
                   200, 2000),                           # nb arbres ensemble
        "max_leaf": trial.suggest_int("max_leaf",
                   15, 255),                             # profondeur arbre
        "min_leaf": trial.suggest_int("min_leaf",
                   5, 50),                               # feuilles min noeud
        "l2":      trial.suggest_float("l2",
                   1e-4, 5, log=True),                   # regularisation poids
    }
    model = HistGradientBoostingRegressor(**params)
    model.fit(X_train, y_train)
    return mean_absolute_error(y_val, model.predict(X_val))
```

Résultats

Le modèle est entraîné sur les données 2020–2025 ($\approx 52\,000$ samples) et validé sur 2026 ($\approx 8\,000$ samples) avec une séparation temporelle stricte, simulant exactement les conditions de la compétition. Voici les résultats obtenus :



Une fois les meilleurs hyperparamètres trouvés, on réentraîne le modèle sur l'intégralité des données 2020–2026 pour enrichir le dataset. On exporte ensuite les trois fichiers `model_temp.pkl`, `model_wind.pkl` et `model_rain.pkl` prêts à être déployés sur ML Arena.

Pipeline complet

$$X \in \mathbb{R}^{20\times 24\times 8} \xrightarrow{\phi(\cdot)} \mathbf{f} \in \mathbb{R}^{98} \xrightarrow{f_\theta} \begin{cases} \hat{T}_{t+6} = T_t + \hat{\Delta}_T \\ \hat{W}_{t+6} = W_t + \hat{\Delta}_W \\ \hat{R}_{t+6} = \max(0,\ \hat{R}) \end{cases}$$

Merci pour votre attention :)

GitHub : `https://github.com/adnanegrb/WeatherNauts`

Rapport : WeatherNauts.pdf